SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Code of Conduct:

I P.Ranjithkumar (192411119)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Ranjithkumar

1. A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.

Answer

Equivalence Class Partitioning (ECP) divides an input domain into classes expected to behave similarly. A representative value from each class is tested. Classes should be mutually exclusive and collectively cover the meaningful input domain. Boundary Value Analysis (BVA) focuses on values at, immediately below, and immediately above limits because off-by-one and comparison errors often occur there.

Errors and root causes

- Valid and invalid values have been mixed into one class, so one representative cannot reliably represent the behaviour of the whole class.
- Classes are not defined from the business rules and permitted input ranges.
- No explicit lower or upper limits have been identified.
- Boundary values such as minimum−1, minimum, minimum+1, maximum−1, maximum, and maximum+1 are missing.
- Test selection is not traceable to requirements, so gaps may remain unnoticed.

Worked example: assume the shopping system accepts quantity from 1 through 10 items per order. This is an illustrative rule; actual limits must come from the requirements.

Class ID	Input range	Type	Representative test	Expected result
EC1	quantity < 1	Invalid low	0	Reject with validation message
EC2	1 ≤ quantity ≤ 10	Valid	5	Accept quantity
EC3	quantity > 10	Invalid high	11	Reject with

				validation message

Boundary test	Input	Expected result	Purpose
B1	0	Reject	Just below minimum
B2	1	Accept	Minimum
B3	2	Accept	Just above minimum
B4	9	Accept	Just below maximum
B5	10	Accept	Maximum
B6	11	Reject	Just above maximum

Diagram: input domain

```
Invalid low        Valid equivalence class (1..10)        Invalid high
----------|=======================================|----------
          0   1   2       ...       9   10   11
            ^ minimum                 ^ maximum
```

Why the correction is better

The revised classes separate expected acceptance from rejection and provide representative coverage of each behaviour. The boundary set checks common defects such as using quantity > 1 instead of quantity >= 1, allowing 11 items, or rejecting exactly 10. Together, ECP reduces redundant tests while BVA targets high-risk edge conditions.

2. A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.

Answer

Random testing chooses inputs without a systematic mapping to requirements, partitions, boundaries, states, or combinations. It can discover unexpected failures, but random input selection does not prove requirement coverage. A large number of random tests may repeatedly exercise ordinary values while missing rare but important edge cases.

Errors and root causes

- Test quantity is incorrectly treated as proof of test coverage.
- Transaction minimums, maximums, daily limits, and cumulative totals were not systematically tested.
- Random values are unlikely to land exactly on important boundaries.
- Possible states (logged in, locked, insufficient funds, pending transaction) and transitions may be omitted.
- Expected results and requirements-based pass/fail oracles may be unclear.

Illustrative requirement: a single transfer must be between ₹1 and ₹50,000, and a customer's daily total must not exceed ₹1,00,000. Confirm these limits from the real specification.

Technique	Example tests	Defects targeted
Equivalence Partitioning	₹0; ₹500; ₹50,001	Invalid low, valid, invalid high
Boundary Value Analysis	₹0, ₹1, ₹2, ₹49,999, ₹50,000, ₹50,001	Off-by-one and limit errors
Decision Table Testing	Valid account? Sufficient balance? Daily limit available?	Incorrect combinations of conditions
State Transition Testing	Active → transfer pending → success/failure; locked account attempts transfer	Illegal or missing state transitions
Pairwise / combinatorial testing	Transfer amount × account status × channel × currency	Interaction defects across factors
Random testing (supplementary)	Seeded random amounts and account states	Unexpected combinations and robustness

Practical correction plan

1. Convert banking requirements into explicit conditions, limits, states, and expected outcomes.
2. Build equivalence classes for valid and invalid transaction amounts and account states.
3. Test every critical numeric boundary, including the daily cumulative total and the transaction immediately over the limit.
4. Use decision tables for combinations such as account active, sufficient balance, beneficiary valid, and limit not exceeded.
5. Use state-transition tests for pending, successful, failed, reversed, and locked transactions.
6. Keep random testing as an additional technique, using reproducible seeds and recording the generated cases.

This combined approach gives measurable requirement and boundary coverage. It is more reliable than claiming high coverage from a test count alone, and it still retains random testing's ability to expose unexpected behaviour.

3. A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.

Answer

A Control Flow Graph (CFG) represents program execution. Nodes represent statements or basic blocks; directed edges represent possible control transfers. Decision nodes have multiple outgoing edges, and loop-back edges must be shown. An incomplete CFG under-represents the program's real execution choices.

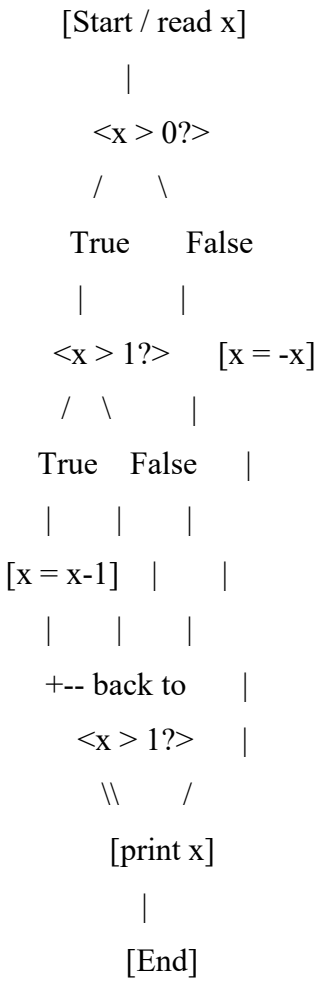
Errors and root causes

- Decision outcomes (true and false) are omitted, hiding possible branches.
- Loop entry, loop body, loop exit, and back edge are not represented.
- Independent paths are not derived from the actual code.
- Tests may execute statements without exercising all edges or meaningful combinations.
- Unreachable code, exception paths, and early returns may not be examined.

Worked example code:

```
read x
if (x > 0):
   while (x > 1):
      x = x - 1
else:
   x = -x
print(x)
```

Corrected CFG



The loop's true edge returns to the loop condition after decrementing x; its false edge exits the loop. The diagram is schematic, but it includes both decision outcomes and the loop-back edge.

Independent paths and test data

Cyclomatic complexity for a single connected CFG is V(G) = E − N + 2P, where E is the number of edges, N is the number of nodes, and P is the number of connected components (normally 1). It estimates the number of linearly independent paths, not the total number of paths when loops can repeat indefinitely.

Path	Route	Example input	Purpose
P1	Start → decision false → else → print → End	x = -3	Exercises false branch
P2	Start → decision true → loop condition false → print → End	x = 1	Exercises true outer branch and zero loop iterations
P3	Start → decision true → loop true → body → loop false → print → End	x = 2	Exercises loop body and loop exit
P4	Start → decision true → loop true repeatedly → exit → print → End	x = 4	Exercises repeated iteration and back edge

For loop-containing programs, complete path enumeration is generally infinite. Use basis-path testing plus loop-specific tests (0, 1, 2, typical, and maximum supported iterations) and test each decision outcome. Better CFG construction makes hidden branches visible and allows tests to be traced to edges and independent paths.

4. A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.

Answer

Statement coverage measures whether every executable statement has run at least once. It does not guarantee that each decision has taken both true and false outcomes. A test can execute all statements while leaving a branch outcome untested, especially when statements are shared after a decision or when a condition is logically wrong.

Example:

```
if (age >= 18)
    allowEntry = true;
logAttempt();
```

A single test with age = 20 executes allowEntry = true and logAttempt(). It gives statement coverage of all executable statements in this small example, but it does not test the false outcome. A defect such as using age > 18 instead of age >= 18 is exposed by age = 18, not by age = 20.

Coverage criterion	What it checks	Limitation
Statement coverage	Every executable statement runs	May miss false branches and logical errors
Branch/decision coverage	Every decision outcome runs at least once	May not exercise every condition combination
Condition coverage	Each atomic Boolean condition is true and false at least once	Does not necessarily cover every decision outcome for compound expressions
Modified Condition/Decision Coverage (MC/DC)	Each condition independently shown to affect the decision outcome	More demanding; commonly used in high-assurance systems
Path coverage	Execution paths through the CFG	Full path coverage is often infeasible with loops

Improved tests

Test	age	Expected decision	Coverage contribution
T1	18	True	Checks inclusive minimum
T2	17	False	Checks false branch
T3	19	True	Checks ordinary valid value

The recommended minimum is branch coverage for each decision, plus BVA where numeric limits exist. For compound Boolean expressions, add condition coverage or MC/DC as appropriate. Path testing can then target important combinations and loops. These criteria provide stronger evidence than statement coverage alone, while coverage percentages should still be combined with requirement-based assertions and code review.

5. A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.

Answer

The two-class design is incorrect because it treats all positive ages alike, even though ages 1 through 100 may be valid while ages greater than 100 are invalid. It also places age 0 in neither negative nor positive numbers, ignores non-numeric inputs, and may overlook decimals or missing values.

Class	Input domain	Type	Representative	Expected result
EC1	age < 0	Invalid	-1	Reject
EC2	0 ≤ age ≤ 100	Valid	35	Accept
EC3	age > 100	Invalid	101	Reject
EC4	Not an integer / non-numeric / blank	Invalid if whole-number age required	'abc' or blank	Reject with clear validation

BVA case	Value	Expected result
Below minimum	-1	Reject
Minimum	0	Accept
Above minimum	1	Accept
Below maximum	99	Accept
Maximum	100	Accept
Above maximum	101	Reject

If the field accepts only whole years, also test 0.5, -0.5, an empty string, whitespace, very large numbers, and malformed text according to the requirements. If age 0 is not acceptable in the business domain, the specification must explicitly change the valid range; tests should follow the agreed rule.

```
Invalid: age < 0 | Valid: 0 through 100 | Invalid: age > 100
-----------------|====================|----------------
      -1         0    1 ... 99   100       101
```

The corrected partitioning separates each distinct expected behaviour, while BVA probes the edges where comparison operators are commonly implemented incorrectly. Together they improve both breadth and precision of validation testing.

6. The supplied question is incomplete: "A developer claims full Condition Coverage for a complex if(A && B …". The answer below interprets the intended expression as if (A && B) and explains how to evaluate the claim.

Answer

For the decision D = A && B, condition coverage requires each atomic condition A and B to take both True and False values at least once. It is different from decision coverage, which requires the overall expression D to evaluate to both True and False.

Test	A	B	A && B	Purpose
T1	True	True	True	Both conditions true; decision true
T2	False	True	False	A false; decision false
T3	True	False	False	B false; decision false

T1 and T2 alone provide condition coverage for A and B (A is true in T1 and false in T2; B is true in both), but B never becomes false. Therefore, two tests are not enough for full condition coverage in this example. T1, T2, and T3 demonstrate each condition as true and false and the decision as true and false.

In languages with short-circuit evaluation, B may not actually be evaluated when A is false. Test T3 is particularly important because it evaluates B as false while A is true. For a safety-critical expression, consider MC/DC: show that changing each condition alone can change the decision outcome while other conditions are held fixed.

```
A ----\
      AND ----> Decision D
B ----/
```

If the original expression contains more conditions (for example, A && B || C), build a truth table and choose tests systematically; do not infer full condition coverage from the number of test cases alone.

7. A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.

Answer

Decision Table Testing maps combinations of conditions to actions. For two independent Boolean conditions—Premium Member and Coupon Applied—there are $2^2 = 4$ combinations. Omitting any combination leaves a possible business rule untested.

Illustrative policy for this example: premium members receive 10% off; a valid coupon gives 5% off; when both apply, discounts are additive (15% total). Real systems must specify whether discounts stack, are capped, or the best discount is selected.

Rule	Premium member?	Valid coupon?	Action / discount	Example on ₹1,000
R1	No	No	No discount	₹1,000 payable
R2	No	Yes	5% coupon discount	₹950 payable
R3	Yes	No	10% member discount	₹900 payable
R4	Yes	Yes	15% combined discount	₹850 payable

Decision table diagram

Premium?	Coupon?	Result
No	No	0% discount
No	Yes	5% discount
Yes	No	10% discount
Yes	Yes	15% discount

The table is exhaustive for these two binary conditions. Additional conditions should be added if required, such as coupon validity, expiry, minimum basket value, product exclusions, maximum discount, or whether the order is already discounted. With n independent Boolean conditions, the unreduced table has 2^n combinations. Rules may only be merged when requirements prove they have identical outcomes.

Testing all four combinations checks interaction logic and helps reveal omitted precedence rules, incorrect stacking, and wrong totals. Include assertions for both discount amount and final payable price, and use monetary rounding rules specified by the application.

8. During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.

Answer

The strategy skips zero iterations and the first invalid count above the maximum. A loop may fail when it should execute zero times, execute once too many, stop one iteration early, or mishandle its final update. Testing only n=1 and n=max does not adequately cover these cases.

For a loop that supports 1 through max iterations, a useful test set is n = 0, 1, 2, typical value, max−1, max, and max+1. If zero iterations are valid, verify the body is skipped. If n=max+1 is invalid, verify validation rejects it safely rather than entering an unsupported loop.

Test count	Expected behaviour	Reason
n = 0	Zero body executions, or explicit rejection if prohibited	Tests empty-loop handling
n = 1	One body execution	Minimum positive iteration
n = 2	Two executions	Detects repeat/update defects
Typical n	Correct repeated processing	Ordinary use
n = max−1	Correct result near upper limit	Near-boundary behaviour
n = max	Correct result at limit	Maximum supported case
n = max+1	Reject or safely handle as specified	Above-limit handling

Nested loops

- For a single loop, test zero, one, two, typical, and boundary iteration counts, subject to the specification.
- For nested loops, first test the inner loop independently, then the outer loop, then combinations such as outer=0, outer=1 with inner=0/1, and small repeated combinations.
- Vary inner-loop counts across outer iterations to detect stale counters and incorrect resets.
- Check termination, loop invariants, array bounds, accumulated totals, and exceptions.
- For very large iteration counts, use risk-based sampling and performance/stress tests rather than attempting every possible count.

  Outer loop: 0, 1, 2, typical, max

  └── Inner loop: 0, 1, 2, typical, max

  Test combinations: (outer=0), (1,0), (1,1), (2,1), (2,2), plus specified limits

This strategy checks entry, repetition, termination, and limit handling. It improves path integrity by exercising the loop's key control-flow transitions, though full path coverage for arbitrary loops is generally impossible because the number of iteration paths can be unbounded.

9. An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.

Answer

Error Guessing uses experience to anticipate likely defects. It is useful as a supplementary technique, but it is subjective and difficult to demonstrate as complete. For a braking system, undocumented assumptions, missed states, and untested limits can create unacceptable safety risks.

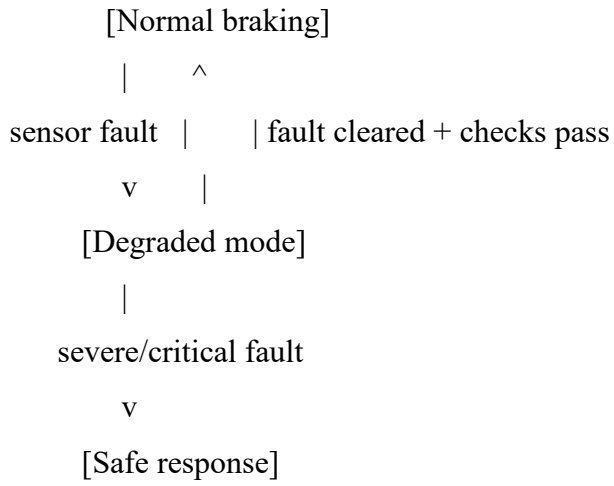
Risks of relying only on error guessing

- Coverage depends on the individual tester's experience and may not be repeatable.
- Requirements and safety constraints may not map to test cases.
- Thresholds such as speed, pedal pressure, temperature, sensor ranges, and response-time limits may be missed.
- Transitions among normal braking, ABS activation, sensor fault, degraded mode, and recovery may be untested.
- Ad-hoc tests may not provide traceability, objective evidence, or repeatable results required by a safety process.

Recommended combination

1. Boundary Value Analysis: identify each specified threshold and test just below, exactly at, and just above it. Example only: if a simulated controller activates a warning at 80 km/h, test 79.9, 80.0, and 80.1 km/h according to the requirement. Do not use this illustrative threshold as an engineering specification.

2. State Transition Testing: define states, events, guards, actions, and permitted transitions. Test both valid and invalid transitions, including fault detection and recovery.



Other transitions (ABS activation, wheel-slip detected, sensor disagreement) must be derived from the system requirements and safety analysis.

Test category	Example test	Expected evidence
BVA lower edge	Sensor input just below/at/above valid minimum	Correct acceptance or fault response
BVA upper edge	Input just below/at/above valid maximum	No overflow; specified fault handling
State transition	Normal → ABS active when wheel slip condition occurs	ABS state and output match specification
Fault transition	Normal → degraded after sensor failure	Fault detected within specified time; safe behaviour
Recovery	Degraded → normal after validated recovery	No premature recovery; diagnostics satisfied
Invalid transition	Attempt recovery while fault persists	Remain in safe/degraded state

For safety assurance, supplement these black-box techniques with requirements traceability, hazard-based testing, code-level coverage where required, hardware-in-the-loop tests, independent review, and verification against applicable safety standards. BVA targets numeric edge failures; state-transition

testing targets sequence and mode failures. Neither alone proves the system safe, but together they provide stronger, auditable evidence than error guessing alone.

10. A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.

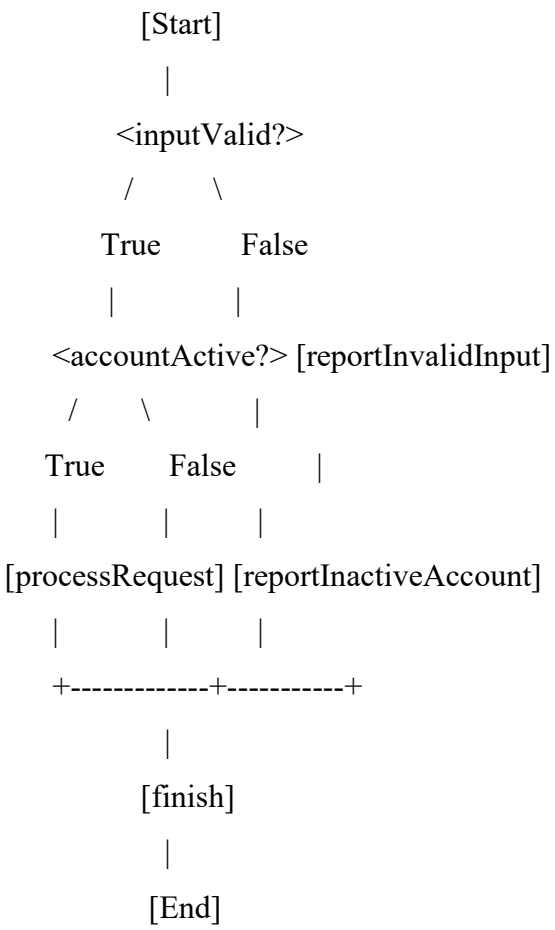
Answer

A CFG must represent every feasible control transfer, including the true and false edges of each if statement, nested else blocks, error handling, returns, and joins. If an else branch is omitted, the graph no longer reflects the program and tests derived from it cannot establish branch adequacy. Statement coverage may still be reported as 100% if tests execute statements on the successful route, but error-path decisions remain untested.

Illustrative code used for the corrected CFG:

```
if (inputValid) {
   if (accountActive) {
      processRequest();
   } else {
      reportInactiveAccount();
   }
} else {
   reportInvalidInput();
}
finish();
```

Corrected CFG



Independent paths and test cases

Path	inputValid	accountActive	Expected route
P1	True	True	processRequest → finish
P2	True	False	reportInactiveAccount → finish
P3	False	Not evaluated / irrelevant	reportInvalidInput → finish

For these two nested decisions, a basis set contains at least three feasible routes. Each decision's true and false outcomes should be exercised: inputValid true/false and, when inputValid is true, accountActive true/false. If the real code contains more nested conditions, exceptions, or additional else-if branches, expand the CFG and path set accordingly.

Correcting the graph makes error-handling paths visible and allows every branch to be linked to a test. Branch coverage is especially important here because a test can execute all success statements yet never exercise invalid input or inactive-account handling. Assertions should verify error codes/messages, side effects, and that processing is not performed after validation fails.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand